

# ATP

## Account Technology Profiler

### Engineering Document

An autonomous, agentic operating system for a Solutions Architect

Local NVIDIA Nemotron-Super-49B | vLLM · 2× H100 (TP=2) | Reasoning Brain + Multi-agent Brainwave  
| No API keys

**Manuel Zelaya**

Solutions Architect

Version 2.0 | June 2026

SHI International Corp. | Solutions Engineering

**CONFIDENTIAL**

# TABLE OF CONTENTS

- 1 Architecture Overview
- 2 Run Model & Launch
- 3 The NeMotron Engine
- 4 The Brain — Intent Routing
- 5 Vault RAG — Retrieval Over Customers
- 6 Live Web Research
- 7 The Brainwave — Multi-Agent Fan-Out
- 8 The Job Engine
- 9 Skills Registry & SEdraw
- 10 ATP Subsystem
- 11 Customer Vault & Soft-Delete
- 12 NeMo Guardrails
- 13 Operations & Scripts
- 14 Security & Data Boundary
- 15 File Map
- 16 Configuration Reference
- 17 Troubleshooting

# 1. ARCHITECTURE OVERVIEW

**ATP — Account Technology Profiler** — is an autonomous, agentic operating system for a Solutions Architect. From a single command box it interprets intent, decomposes goals, dispatches parallel work, reasons over a per-customer technology profile, and synthesizes cited deliverables. Its defining property is that it runs on a **self-hosted NVIDIA model**: `nvidia/Llama-3.3-Nemotron-Super-49B-v1.5` served by **vLLM** across **two H100 NVL GPUs** at `tensor-parallel = 2` — with **no API keys, no cloud LLM, and no customer-data egress**.

ATP descends from ATLAS, whose inference substrate was Microsoft 365 Copilot driven through a browser (Selenium + a pool of Chrome agents). ATP replaces that substrate wholesale with local Nemotron inference. The Copilot/Chrome engine is hard-disabled (`ATLAS_COPILOT_DISABLED=1`); the browser pool, M365 calendar ingestion, and Selenium agents are retired. The application shell, the concurrent job engine, the vault, the skill registry, and the agentic patterns were preserved and re-grounded on Nemotron.

## Request Lifecycle

| Layer        | Component                                              | Role                                                                                                                                   |
|--------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1 UI         | <code>atlas/web/index.html · run_atlas_linux.py</code> | Local single-page console (Chrome app-mode); pull model                                                                                |
| 2 API        | <code>atlas/web/{server,api}.py</code>                 | <code>POST /api/&lt;method&gt; → Api.&lt;method&gt;</code> ; returns a job id / JSON                                                   |
| 3 Guardrails | <code>atlas/brain/guardrails.py</code>                 | Optional input rail on <code>command()</code> (off by default)                                                                         |
| 4 Brain      | <code>atlas/brain/nemotron_brain.py</code>             | Nemotron classify → <code>run_skill</code> / <code>navigate</code> / <code>rag</code> / <code>research</code> / <code>brainwave</code> |
| 5 Jobs       | <code>atlas/jobs/manager.py</code>                     | Worker pool; runs the decision as cancellable jobs                                                                                     |
| 6 Engine     | <code>atlas/engine/nemotron.py · websearch.py</code>   | Local Nemotron client + live web research                                                                                              |
| 7 Stores     | <code>vault/ + atlas/store/* + atlas/atp/*</code>      | Customers, artifacts, ATP, settings, recyclebin                                                                                        |

## Key Design Decisions

- **Sovereign local inference.** Nemotron-49B runs on the user's own 2× H100s via vLLM; the same OpenAI-compatible client is NIM-ready (one env var to repoint).
- **Everything is an artifact.** Every deliverable and brain interaction is logged, in-app readable, searchable, and (where applicable) transferable to a customer.
- **Non-destructive.** Deletes are PIN-guarded soft-deletes to `vault/recyclebin/`; ATP regeneration never clobbers hand-edits (pin-and-merge).
- **Concurrent & cancellable.** All real work is a background job; jobs survive navigation/restarts; the **X** recursively cancels a job tree and aborts in-flight generation.
- **Self-aware & auditable.** Every command yields a reasoning receipt in Brainwave History (decision, engine-aware agent trace, rationale, self-check).

# 2. RUN MODEL & LAUNCH

ATP launches from `run_atlas_linux.py` with `ATLAS_COPILOT_DISABLED=1`. It backfills trip-report artifacts, starts the API server, and blocks (the UI is opened separately as a Chrome app-mode window, e.g. via the ATP desktop shortcut). The cron scheduler and calendar auto-refresh are intentionally NOT started (they drove M365/Chrome).

`run_atlas_linux.py`

```

ATLAS_COPILOT_DISABLED=1          # hard backstop: no browser may launch
artifacts.backfill_trip_reports() # every trip-report .md → artifact
Api()                             # the facade the UI calls
ThreadingHTTPServer 127.0.0.1:8080 (atlas/web/server.py)
threading.Event().wait()          # server runs in a daemon thread

```

The UI is a **pull model**: the page polls jobs/artifacts (~1.5 s, no-store) and re-renders. Long work is always a background job — it never blocks the UI.

### vLLM serving (the model)

vLLM is managed out-of-process by `~/vllm_service_manager.sh` in a tmux session named `ai`. Launch flags:

```

VLLM_USE_FLASHINFER_SAMPLER=0 python -m vllm.entrypoints.openai.api_server \
--model nvidia/Llama-3.3-Nemotron-Super-49B-v1_5 --trust-remote-code \
--host 0.0.0.0 --port 8000 \
--served-model-name Llama-3.3-Nemotron-Super-49B-v1_5 \
--tensor-parallel-size 2 --gpu-memory-utilization 0.85 \
--max-model-len 8192 --enforce-eager

```

- **TP=2** shards the 49B weights across both H100 NVL cards (~82 GB resident per card).
- **8192 context.** The 32K FlashInfer path needs a CUDA toolkit (nvcc) not installed here; `VLLM_USE_FLASHINFER_SAMPLER=0` uses the native sampler at 8K.
- Manager menu: `status · start · graceful stop · restart · quick-recover · GPU topology · attach · tail logs`.

## 3. THE NEMOTRON ENGINE

`atlas/engine/nemotron.py` is the single inference client. It is ENV-driven so the same code targets local vLLM or an NVIDIA NIM:

```

LLM_BASE_URL http://localhost:8000/v1      # or a NIM endpoint
LLM_MODEL    Llama-3.3-Nemotron-Super-49B-v1_5
LLM_API_KEY   dummy                      # vLLM ignores it; SDK requires a value

```

```

complete(prompt, *, system='detailed thinking off', temperature=0.6, top_p=0.95,
          max_tokens=None, retries=2, on_log=None, should_cancel=None) -> str

```

### Nemotron quirks handled

- **<think> block.** The model always emits `<think>...</think>` reasoning; `strip_think()` removes it before returning.
- **Sampling.** Greedy (temp 0) loops; NVIDIA's recommended temp 0.6 / top\_p 0.95 is the default.
- **Token budgeting.** The live `max_model_len` is read from `/v1/models`; when `max_tokens` is `None` it is budgeted as `window – est(prompt)×1.5 – margin (capped)` so prompt + reasoning + answer fit 8K.

### Streaming cancellation (free the GPU)

When `should_cancel` is supplied, the request is streamed and the flag is polled between chunks; on cancel the stream is closed (vLLM stops generating) and a `Cancelled` exception is raised. This is the local-model analog of closing a browser tab — the worker unwinds and the GPU is freed immediately.

### Context-overflow self-correction

If vLLM rejects a request for exceeding 8192 tokens, the 400 error reports the real prompt size ('N in the messages'). `complete()` parses N and retries with `max_tokens` shrunk to fit — so a mis-estimated prompt self-corrects instead of failing.

## 4. THE BRAIN — INTENT ROUTING

`atlas/brain/nemotron_brain.py` + `JobManager._brain_route_nemotron` route a free-text command through one Nemotron classification call into five actions.

| Action    | Trigger (example)                         | Behavior                                            |
|-----------|-------------------------------------------|-----------------------------------------------------|
| run_skill | "strategic briefing for Acme"             | Dispatch the skill job for the resolved customer    |
| navigate  | "create a network diagram for Acme"       | Tell the user to open Skills → the interactive tool |
| rag       | "which customers use Palo Alto?"          | Cited retrieval over the Customer Vault (\$6)       |
| research  | "research the latest Palo Alto firewalls" | Live web-research agent (\$7)                       |
| brainwave | "is Verkada a good fit for Acme?"         | Multi-agent fan-out + self-check (\$8)              |

### Skill grounding without per-skill rewrites

The Copilot-era data skills (`strategic_briefing`, `account_360`, `battlecard`, `whitespace`, `follow_up`, `environment_topology`) call an `ask()` that assumed M365 grounding. On Nemotron the job runner rebuilds that closure as `ask = lambda p: nemotron.complete(saved_context(customer) + p)` — the customer's saved ATP profile + recent trip reports (hard-capped to fit 8K) are prefixed onto the skill's own prompt. The skills produce real grounded JSON/markdown/diagrams with **zero per-skill code changes**.

### Reasoning receipts (engine-aware)

Every interaction writes a Brainwave-History receipt: the decision line ("decided **run\_skill** → `strategic_briefing`"), grounding ("routed on **local Nemotron**"), an agent trace table whose Engine column reads `Nemotron-49B (vLLM·2×H100)`, and a model-generated rationale (the 'why'). The receipt machinery is engine-aware — no Copilot/Chrome/Selenium wording in the local build — and a `finalize_receipt` job enriches it after the spawned work settles.

## 5. VAULT RAG — RETRIEVAL OVER CUSTOMERS

`nemotron_brain.rag_answer()` answers questions across one, several, or **all** customers.

- **Scope + terms.** A quick Nemotron parse decides scope (named vs all) and the search terms, with a deterministic fallback; names resolve to vault slugs via `resolve_customer`.
- **Deterministic retrieval (cheap, scales to ~84 customers).** Per customer it scans `data_library.json` (`vendor/product/category/notes`), `trip_reports/*.md` (line snippets), `overview.json`, `vision/network *.xlsx` cell text, and the customer's artifacts — collecting bounded `{customer, source, detail}` hits.
- **Cited synthesis.** Nemotron writes a markdown answer that cites where each fact was found, treating the `## <Customer>` header as authoritative (so leftover names inside content don't misattribute), and says plainly when nothing matched.

The customer scan loop and synthesis honor should\_cancel, so a long 'all customers' query stops promptly when the job is cancelled.

## 6. LIVE WEB RESEARCH

atlas/engine/websearch.py (httpx + lxml; no API key) is the only outbound path. search() queries DuckDuckGo (lite, then html), unwraps redirect links, and drops results whose host matches a Guardrails-blocked domain. fetch() extracts readable page text. research\_brief() returns a compact numbered SOURCES block (top results + the first few fetched pages). If egress fails, it returns empty and the agent falls back to model knowledge with a logged note.

## 7. THE BRAINWAVE — MULTI-AGENT FAN-OUT

For "is <product> a good fit for <customer>?", JobManager.\_brainwave\_nemotron orchestrates a parallel agent graph:

- **Fan out two agents in parallel** (each a real job on its own worker → genuine concurrent inference batched on the H100s): ■ bw\_research (live web research → Nemotron brief) ■ ■ bw\_cust\_saved (Nemotron over the customer's saved data).
- **Merge** both into an ephemeral wave.md.
- ■ **synthesis agent** — a grounded verdict (Verdict / Where it fits / Risks & objections / Recommended next step), citing the customer's real stack.
- ■ **self-check agent** — a separate Nemotron pass fact-checks the verdict against the evidence, flagging unsupported claims with a confidence rating ('the brain checks its own work').
- **Audit receipt** — a brainwave artifact records the parallel-agent trace, the verdict, and the self-check in Brainwave History.

Cancelling the brainwave recursively cancels its sub-agents, skips synthesis/self-check, and marks the in-progress artifact '■ cancelled'. The M365 live-tenant freshness agent from the Copilot era is dropped (no tenant).

## 8. THE JOB ENGINE

atlas/jobs/manager.py runs a Job queue on a worker pool (POOL\_SIZE = 4). Each request is a Job mirrored to vault/jobs.json (survives restarts).

### Recursive, abortable cancellation

submit(kind, ctx, ..., parent=) links sub-jobs into a tree. cancel(jid) walks the job + all descendants and, for each, sets the per-job cancel flag (polled by the streaming nemotron.complete to abort mid-generation and free the GPU). A cancelled brainwave stops its research/customer sub-agents, skips downstream work, and cleans its artifact.

### Job kinds

|             |                                 |
|-------------|---------------------------------|
| brain_route | the Brain (route + dispatch)    |
| brainwave   | multi-agent fan-out facilitator |
| bw_research | ■ web research sub-agent        |

|                  |                                                    |
|------------------|----------------------------------------------------|
| bw_cust_saved    | ■ customer-infrastructure sub-agent                |
| web_research     | standalone research action (§6)                    |
| docx_network     | Word .docx → Nemotron → network intake → map       |
| vision_generate  | text → combined current/future vision .drawio      |
| atp_generate     | map-reduce ATP profile build                       |
| <skill kinds>    | strategic_briefing, battlecard, ... (grounded)     |
| finalize_receipt | enriches a brainwave receipt after children settle |

## 9. SKILLS REGISTRY & SEDRAW

Skills are `Skill.run(ctx, ask, log) → SkillResult`. For the demo the Skills page shows only the interactive SEdraw tools; the rest remain registered and Brain-runnable (a single `_HIDDEN_SKILLS` filter in `api.skills()` — nothing is deleted).

### SEdraw — Network Diagrams

- Excel intake (network\_input/\*.xlsx; sites → categories with Qty×Model + connections) → NetworkTopologyGenerator → multi-site .drawio + a self-contained diagrams.net viewer.
- Full in-GUI editor; ■ Create New blank template; per-file ■ PIN soft-delete.
- **Convert from DOCX**. docx\_to\_network.py extracts a Word intake (python-docx: paragraphs + tables), Nemotron maps it onto the 23 valid network categories (best-effort, omits what doesn't fit), writes the intake .xlsx, and generates the map — a cancellable docx\_network job. The .xlsx is editable + regenerable.

### SEdraw — Vision Boards (Vision Studio)

- Two free-text boxes (current / future state) → Nemotron (vision\_ai.py) → one combined diagram (current top, future bottom) → VisionBoardGenerator .drawio + viewer.
- Runs as a background vision\_generate job (tasks bar + artifact); history loads older sessions; full editor; backend .xlsx hidden, named customer+date; per-session ■ soft-delete.

## 10. ATP SUBSYSTEM

atlas/atp/\*.py builds and maintains the per-customer technology profile from filed trip reports.

- extract.generate\_local(ask, customer, log)** — map-reduce: chunk the trip-report text to fit 8K, run TECH / OVERVIEW / CONTACTS prompts per chunk concurrently (thread pool), then reduce (merge technologies, clean+merge contacts, synthesize overview). No browser.
- generations.py** — each '■ Generate' is an immutable timestamped snapshot under account\_technology\_profile/generations/<id>; the live flat files are the editable working copy; pin-and-merge means hand-edits survive regeneration. Per-snapshot ■ delete (live 'Current' protected).
- data\_library / profile\_store / recency / topology\_html** — structured tech inventory (vendor, product, category, deployment status, mention\_count, last\_seen), overview + contacts stores, recency scoring, rendered topology.

## 11. CUSTOMER VAULT & SOFT-DELETE

|                             |                                                   |
|-----------------------------|---------------------------------------------------|
| vault/customers/<slug>/     |                                                   |
| trip_reports/               | filed .md trip reports                            |
| account_technology_profile/ | data_library.json · overview.json · contacts.json |

```
generations/<id>/          immutable snapshots
network_input/  network_output/  SEdraw network .xlsx / .drawio
vision_input/   vision_output/   SEdraw vision .xlsx / .drawio
artifacts/      filed deliverables
```

Customer display names in the demo vault are anonymized (folder/display only). All deletes — a customer, an artifact, a vision board, a network diagram, or a topology generation — are **PIN-guarded soft-deletes** that move the file(s) to vault/recyclebin/ (they vanish from the UI but are never erased). The 6-digit System PIN (Settings) is required; if unset, deletion is blocked. One shared frontend pinDeleteModal + matching PIN-checked endpoints (delete\_customer/artifact/generation, sedraw\_vision\_delete, sedraw\_network\_delete).

## 12. NEMO GUARDRAILS

A NeMo Guardrails input self-check rail on the command box, running on the local Nemotron. **OFF by default, fail-open** (any error → allow), with a native fallback.

### Three-tier evaluation

- **1. Deterministic denylist** — blocked words (instant, no model call).
- **2. NeMo Guardrails LLMRails** — the real 'self\_check\_input' rail on local Nemotron (OpenAI provider → LLM\_BASE\_URL).
- **3. Native fallback** — a tiny Nemotron yes/no classifier if the package can't load.

### Levers (settings.json) & wiring

- master on/off · keep on-topic · block unsafe · block prompt-injection · strictness (lenient/balanced/strict) · blocked words · blocked research domains.
- Changing levers regenerates guardrails\_config/prompts.yml and rebuilds the cached rail.
- Api.command() calls check\_input first; a block returns a refusal (no job) and records to a shared cross-process log vault/guardrails\_blocks.json.
- guardrails\_config/: config.yml (rails + model), prompts.yml (self\_check\_input), rails.co (refusal message).  
Input self-check only — no Colang dialog flows / embedding models (light, fully offline).

### Standalone Control Panel

guardrails\_panel.py is a separate stdlib web app on :8090 (own process, own Desktop shortcut) that reads/writes the same guardrails\_\* settings — so it governs the live rail — but cannot crash ATP. It exposes the levers, a live Test box (ALLOW/REFUSE + reason + which engine ran), and a Recent-blocks log that reads the shared file and auto-refreshes every 4 s.

## 13. OPERATIONS & SCRIPTS

| Script                      | Purpose                                                                                          |
|-----------------------------|--------------------------------------------------------------------------------------------------|
| demo_health.sh              | Pre-demo board: 2xH100, weights resident (TP=2), vLLM health, live generation (latency/tok-s), A |
| guardrails.sh on off status | Terminal toggle for the command-box rail (next command; no restart).                             |
| guardrails_panel.sh         | Launch the Guardrails Control Panel (:8090).                                                     |
| launch_guardrails_panel.sh  | Desktop-shortcut launcher (starts panel if down + opens app window).                             |

|                           |                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------|
| run_atlas_linux.py        | Launch the ATP app server (Copilot/Chrome disabled).                                 |
| ~/vllm_service_manager.sh | vLLM lifecycle in tmux 'ai': status/start/stop/restart/recover/topology/attach/logs. |

## 14. SECURITY & DATA BOUNDARY

- **Sovereign, local inference.** The model runs on the user's own 2×H100s. No API keys, no cloud LLM, no customer-data egress.
- **One opt-in network path.** The web-research agent makes outbound DuckDuckGo/page requests — only the query leaves the box, only when the user runs research/brainwave; domains can be denylisted.
- **PIN-guarded, non-destructive.** All deletes are soft-deletes to recyclebin/, gated by the System PIN.
- **Guardrails.** The command box can be governed (topical/safety/injection + denylists) with a full audit log.
- **NIM-ready.** Repointing to an NVIDIA NIM (local or hosted) is a single env var (LLM\_BASE\_URL).

## 15. FILE MAP

```
~/ATLAS/  
  run_atlas_linux.py · config.py  
  atlas/  
    web/   server.py · api.py · index.html  
    brain/ nemotron_brain.py · guardrails.py · guardrails_config/ ·  
           brainwave.py · capabilities.py · rules.py · router.py (legacy)  
    engine/ nemotron.py · websearch.py · pool.py · copilot.py (disabled)  
    jobs/   manager.py · scheduler.py  
    skills/ base.py + skills · peoplereources/  
    sedraw/ network_*.py · drawio_generator.py · vision_ai.py · docx_to_network.py · service.py  
    atp/    extract · generations · recency · topology_html · data_library · profile_store  
    store/  vault · customers · artifacts · settings · cron · people  
  guardrails_panel.py · demo_health.sh · guardrails.sh · guardrails_panel.sh  
  ATP_ARCHITECTURE.md/.drawio · GUARDRAILS_ARCHITECTURE.drawio · NEMOTRON_ARCHITECTURE.md  
  ATP_Engineering_Document.pdf · ATP_User_Guide.pdf  
  vault/   (runtime data + recyclebin/ + guardrails_blocks.json + jobs.json)  
~/vllm_service_manager.sh
```

## 16. CONFIGURATION REFERENCE

| Key / env                         | Default                           | Purpose                                       |
|-----------------------------------|-----------------------------------|-----------------------------------------------|
| LLM_BASE_URL                      | http://localhost:8000/v1          | vLLM (or NIM) endpoint                        |
| LLM_MODEL                         | Llama-3_3-Nemotron-Super-49B-v1_5 | Served model id                               |
| ATLAS_COPILOT_DISABLED            | 1                                 | hard-disable the legacy Copilot/Chrome engine |
| TP_SIZE / MAX_MODEL_LEN           | 2 / 8192                          | tensor-parallel · context window              |
| POOL_SIZE                         | 4                                 | concurrent job workers                        |
| guardrails_enabled                | false                             | command-box input rail                        |
| guardrails_strictness             | balanced                          | lenient / balanced / strict                   |
| guardrails_blocked_terms/_domains | "                                 | command denylist / research source filter     |

|            |   |                                     |
|------------|---|-------------------------------------|
| system_pin | “ | 6-digit PIN required for any delete |
|------------|---|-------------------------------------|

## 17. TROUBLESHOOTING

| Symptom                              | Cause & Fix                                                                                           |
|--------------------------------------|-------------------------------------------------------------------------------------------------------|
| Model down / app errors on inference | Run ./demo_health.sh; if red, ./demo_health.sh --recover (heals vLLM only if down). Check ~/vllm      |
| 400 'maximum context length' in logs | Prompt + max_tokens exceeded 8K; complete() self-corrects and retries. Persistent → the input is      |
| Raw <think> in a rationale/output    | max_tokens too small truncated the JSON before </think>; budgets were raised + a defensive str        |
| Guardrails blocked a legit command   | Lower strictness or clear blocked words in the panel; or toggle the master off (./guardrails.sh off). |
| A block didn't show in the panel     | Blocks are a shared file (vault/guardrails_blocks.json); the panel auto-polls every 4 s — reload if n |
| DOCX convert produced few sites      | Best-effort: prose/blank tables don't map; edit the generated .xlsx in the Net editor and regenerate  |
| A delete asks for a PIN              | Set a 6-digit System PIN in Settings; deletes are PIN-guarded soft-deletes to recyclebin/.            |